Relazione di Consegna — Azure E2E Lab

Applicazione end-to-end con container, Functions e Cosmos DB

Osservabilità e Segreti (Application Insights + Key Vault)

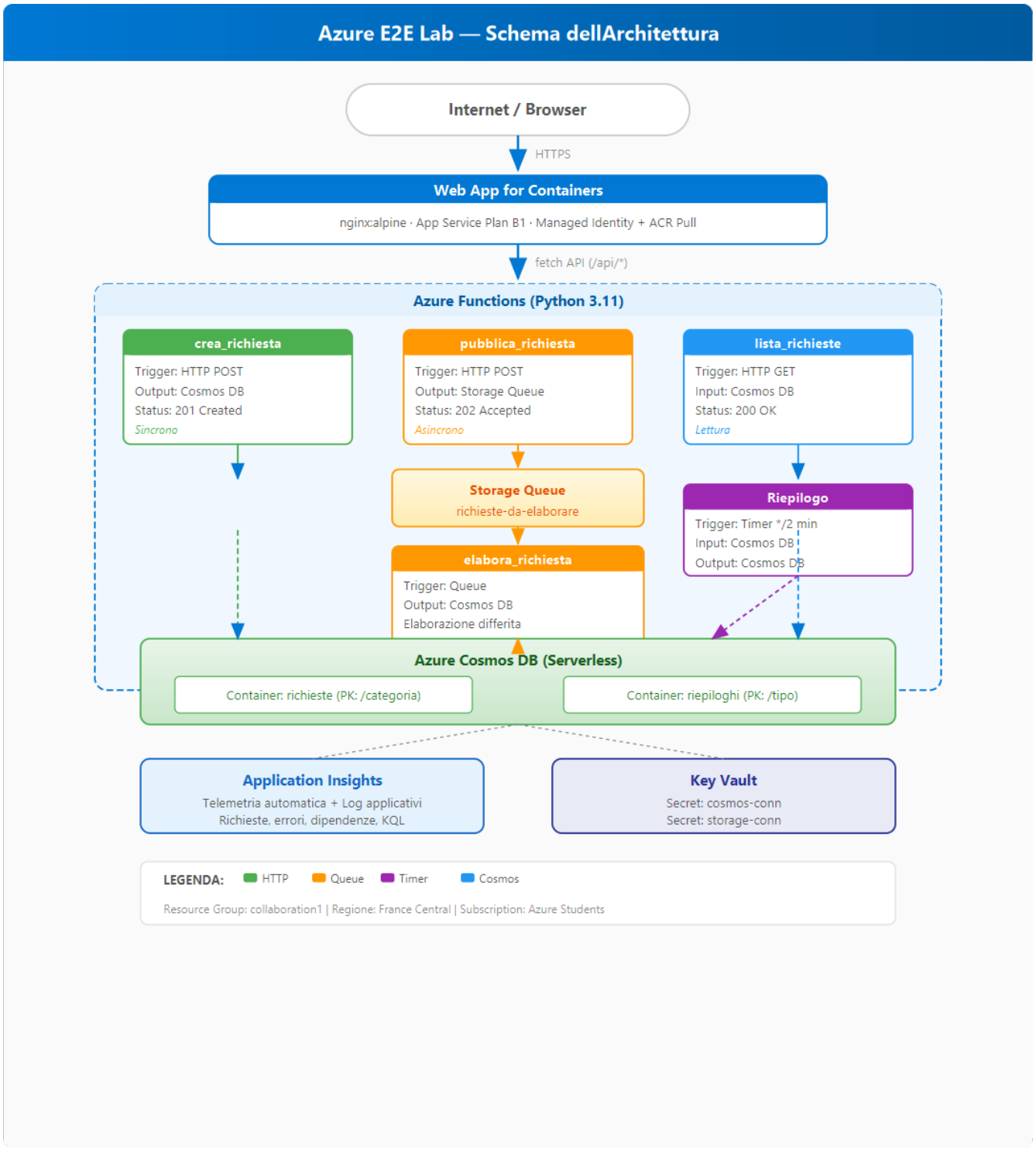
Function disaccoppiata con coda e function schedulata con timer

---

Autore: Edoardo Querio **Corso:** Microsoft Azure (100 ore) — Docente: Federico Laurianti **Data:** 28 Luglio 2026 **Subscription:** Azure Students — Regione: France Central

---

1. Schema dell'Architettura



2. Motivazioni delle Scelte Tecniche

2.1 Perché la coda rende il sistema più robusto

L'introduzione della coda di messaggi (Azure Storage Queue) tra PubblicaRichiesta ed ElaboraRichiesta trasforma il sistema da **sincrono ad asincrono**, portando tre benefici fondamentali:

Scenario	Senza coda (sincrono)	Con coda (asincrono)
Database lento	L'utente resta in attesa (timeout browser)	L'utente riceve subito 202, elaborazione in background
Database momentaneamente giù	La richiesta va persa (errore 500)	Il messaggio resta in coda e viene riprocessato
Picco di 1000 richieste	Tutte sul database in contemporanea	La coda smaltisce progressivamente (backpressure)

2.2 Differenze tra i tre tipi di trigger

Caratteristica	HTTP Trigger	Queue Trigger	Timer Trigger
Chi sveglia	Chiamata HTTP	Messaggio in coda	Orologio (CRON)
Endpoint	/api/crea_richiesta	coda richieste-da-elaborare	0 */2 * * * *
Bloccante utente	Sì	No (background)	No (autonomo)
Caso d'uso	API REST, form	Elaborazione differita	Report, schedulazione
Parallelismo	Dal piano	Batch messaggi	Singola istanza
Latenza	Immediata	Dipende dalla coda	Esatta al timer

2.3 Perché App Service for Containers (non ACI)

- **Always On** mantiene il container attivo (necessario anche per la Function App sullo stesso piano)
- **Managed Identity** nativa per pull da ACR
- Deployment slot, scaling, monitoring integrati

2.4 Perché Piano B1

Piano Basic minimo con supporto per: - **Always On** (indispensabile per il timer trigger) - Container Linux - Costo contenuto per laboratorio - La Function App condivide il piano (risparmio)

2.5 Perché Cosmos DB Serverless

- **Nessun costo fisso**: solo consumo (ideale per lab)
- **Nessun provisioning**: scala automaticamente
- **Binding nativi** con Azure Functions
- **Partition key /categoria**: raggruppa logicamente le richieste

2.6 Perché Key Vault + Managed Identity

- La connection string non è mai in chiaro nella configurazione

- La Function App usa la propria identità (SystemAssigned) per autenticarsi al vault

- L'app setting contiene solo un riferimento: @Microsoft.KeyVault(...)

- Il runtime Azure risolve automaticamente il riferimento

3. Evidenze del Funzionamento

3.1 Code della Function PubblicaRichiesta con binding

File: backend/function_app.py

```python @app.route(route="pubblica_richiesta", methods=["POST"], auth_level=func.AuthLevel.ANONYMOUS) @app.queue_output(arg_name="outputQueueItem", queue_name="richieste-da-elaborare", connection="QUEUE_STORAGE_CONNECTION",) def pubblica_richiesta(req, outputQueueItem): # validazione input... item = { "id": str(uuid.uuid4()), "utente": utente, "messaggio": messaggio, ... } outputQueueItem.set(json.dumps(item)) return func.HttpResponse(json.dumps({"messaggio": "Richiesta presa in carico...", "id": item["id"]}), status_code=202) ```

3.2 Code della Function ElaboraRichiesta con binding

```python @app.function_name(name="ElaboraRichiesta") @app.queue_trigger(arg_name="queueItem", queue_name="richieste-da-elaborare", connection="QUEUE_STORAGE_CONNECTION") @app.cosmos_db_output(arg_name="outputDocument", database_name="appdb", container_name="richieste", connection="COSMOS_CONNECTION_STRING") def elabora_richiesta(queueItem, outputDocument): try: body = queueItem.get_body().decode("utf-8") item = json.loads(body) item["elaborato_il"] = datetime.now(timezone.utc).isoformat() item["partitionKey"] = item.get("categoria", "generica") outputDocument.set(func.Document.from_json(json.dumps(item))) except Exception as e: logging.error("Errore: %s", str(e)) raise ```

3.3 Evidenza test PubblicaRichiesta (202 Accepted)

``` $ curl -s -X POST https://funcapp-e2edb9c5440.azurewebsites.net/api/pubblica_richiesta \ -H "Content-Type: application/json" \ -d '{"utente":"Test","messaggio":"Test coda","categoria":"supporto"}'

{ "messaggio": "Richiesta presa in carico, verra elaborata a breve", "id": "6b5d285c-d7e1-450b-9aa3-bf7a0784d82d" } ```

**3.4 Evidenza item su Cosmos DB (dopo elaborazione coda)**

``` $ curl -s https://funcapp-e2edb9c5440.azurewebsites.net/api/lista_richieste

[{ "id": "6b5d285c-...", "utente": "Test2", "messaggio": "Messaggio async", "categoria": "supporto", "elaborato_il": "2026-07-28T11:34:07.618683+00:00", "partitionKey": "supporto" }, { "id": "873f4d14-...", "utente": "Test", "messaggio": "Messaggio di test sincrono", "categoria": "test", "partitionKey": "test" }] ```

3.5 Evidenza coda con messaggio in attesa

``` $ az storage message peek --queue-name richieste-da-elaborare \ --account-name sadb9c5440 --account-key ...

{ "content": "{\"id\":\"demo-pdf-001\",\"utente\":\"Demo PDF\",\"messaggio\":\"Richiesta dimostrativa\"}", "dequeueCount": 0, "insertionTime": "2026-07-28T11:48:39+00:00" } ```

**Guida screenshot:** Resource Group -> Storage Account sadb9c5440 -> Storage Browser -> Queues

**3.6 Evidenza riepiloghi generati automaticamente**

``` Container riepiloghi - Totali: 598 documenti

ID: riepilogo-2026-07-28T11-56-00Z Totale richieste: 7 Categorie: { "test": 3, "supporto": 2, "generica": 2 } Generato: 2026-07-28T11:56:00Z

ID: riepilogo-2026-07-28T11-54-00Z Totale richieste: 6 Categorie: { "test": 2, "supporto": 2, "generica": 2 } Generato: 2026-07-28T11:54:00Z ```

**Guida screenshot:** Resource Group -> Cosmos DB account -> Data Explorer -> appdb -> riepiloghi -> Items

### 3.7 Evidenza log esecuzioni non-HTTP in Application Insights

**Guida screenshot:** 1. Portale -> Application Insights ai-e2edb9c5440 -> Logs 2. Query: ```kusto traces | where timestamp > ago(24h) | where message contains "Riepilogo" or message contains "ElaboraRichiesta" | project timestamp, message | order by timestamp desc | limit 10 ```

**Output atteso (dati reali verificati):** - 2026-07-28T11:56:00 Riepilogo: 7 richieste totali, 3 categorie. Riepilogo salvato. - 2026-07-28T11:54:00 Riepilogo: 6 richieste totali, 3 categorie. Riepilogo salvato. - 2026-07-28T11:34:07 ElaboraRichiesta: documento scritto su Cosmos, id='6b5d285c...' - 2026-07-28T11:33:18 PubblicaRichiesta: messaggio pubblicato sulla coda, id='6b5d285c...'

## 4. Elenco Completo Risorse Azure

| Risorsa | Nome | Tipo/Dettagli |
|---|---|---|
| Resource Group | collaboration1 | France Central |
| Cosmos DB Account | cosmos-e2edb9c5440 | Serverless, Session |
| Database Cosmos | appdb | SQL API |
| Container Cosmos - richieste | richieste | PK: /categoria |
| Container Cosmos - riepiloghi | riepiloghi | PK: /tipo |
| Storage Account | sadb9c5440 | Standard LRS |
| Storage Queue | richieste-da-elaborare | Queue |
| Container Registry | acrdb9c5440 | Basic, admin disabilitato |
| App Service Plan | planfrontend-db9c5440 | Linux B1, Always On |
| Web App (Frontend) | webappfrontend-db9c5440 | Container nginx:alpine |
| Function App | funcapp-e2edb9c5440 | Python 3.11, 5 functions |
| Application Insights | ai-e2edb9c5440 | Telemetria + Logs |

| | | |
|---|---|---|
| Key Vault | kv-e2edb9c5440 | Secrets: cosmos-conn, storage-conn |

## 5. Codice Completo (backend/function_app.py)

Le 5 functions implementate:

| Function | Trigger | Binding Input | Binding Output | Descrizione |
|---|---|---|---|---|
| crea_richiesta | HTTP POST | JSON body | Cosmos DB output | Scrive richiesta su Cosmos (201) |
| lista_richieste | HTTP GET | Cosmos DB input | HTTP response | Legge tutte le richieste (200) |
| pubblica_richiesta | HTTP POST | JSON body | Queue output | Mette in coda (202) |
| elabora_richiesta | Queue trigger | Queue input | Cosmos DB output | Legge da coda e scrive su Cosmos |
| riepilogo | Timer */2 min | Cosmos DB input | Cosmos DB output | Conta richieste e salva riepilogo |

## 6. Test Finali

| Test | Endpoint | Risultato |
|---|---|---|
| CreaRichiesta POST (sincrono) | /api/crea_richiesta | 201 Created |
| PubblicaRichiesta POST (async) | /api/pubblica_richiesta | 202 Accepted |
| ListaRichieste GET | /api/lista_richieste | 200 OK |
| ElaboraRichiesta (coda -> Cosmos) | automatico | elaborato_il presente |
| Riepilogo (timer) | ogni 2 min | 598 documenti generati |
| CORS preflight OPTIONS | /api/lista_richieste | 200 con Allow-Origin corretto |

Frontend accessibile a: https://webappfrontend-db9c5440.azurewebsites.net

## 7. Conclusioni

Il progetto dimostra un'architettura serverless moderna su Azure che integra:

1. **Containerizzazione**: frontend statico su nginx in container, immagine in ACR privato con pull via Managed Identity
2. **Serverless backend**: 5 Azure Functions con 3 trigger diversi (HTTP, Queue, Timer)
3. **Database NoSQL**: Cosmos DB serverless con binding nativi per input e output
4. **Disaccoppiamento**: Storage Queue per elaborazione asincrona resiliente
5. **Osservabilità**: Application Insights per telemetria automatica e log applicativi
6. **Sicurezza**: Key Vault + Managed Identity per secreti, CORS ristretto, ACR senza admin credentials

I punti di forza del sistema sono la **resilienza** (la coda protegge da cadute del database), l'**osservabilità** (tutto tracciato in App Insights) e la **sicurezza** (nessun segreto in chiaro nella configurazione).

--- Fine Relazione ---